

Benefits of Azure Bicep

Why Bicep exists, what it gives you over raw ARM JSON, the full syntax breakdown, modules, and how it fits into a real deployment pipeline.

ARM JSON vs. Bicep — same resource, side by side

Bicep compiles 1:1 to ARM JSON. The end result is identical — the difference is purely in readability and developer experience.

ARM JSON

```
{
  "$schema":
    "https://schema.management.azure.com/
      schemas/2019-04-
01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "storageAccountName": {
      "type": "string"
    }
  },
  "resources": [
    {
      "type":
        "Microsoft.Storage/storageAccounts",
      "apiVersion": "2023-01-01",
      "name": "[parameters('storageAccountName')]",
      "location": "[resourceGroup().location]",
      "sku": { "name": "Standard_LRS" },
      "kind": "StorageV2"
    }
  ]
}
```

Bicep

```
param storageAccountName string

resource sa
  'Microsoft.Storage/storageAccounts@2023-
01-01' = {
  name: storageAccountName
  location: resourceGroup().location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}
```

Six key benefits of Bicep

01

Simpler syntax

No quotes on property names, no nested bracket expressions, no `$schema` boilerplate. The same resource takes roughly half the lines in Bicep vs. ARM JSON.

02

Type safety at author time

The Bicep compiler catches type mismatches, missing required properties, and invalid values before the template ever hits Azure — fail fast, not at deploy time.

03

Modules

Break large templates into reusable `.bicep` files called modules. Each module has its own parameters and outputs — compose complex environments from tested building blocks.

04

Implicit dependency detection

Reference one resource inside another and Bicep automatically infers the deployment order — no manual `dependsOn` needed in most cases.

05

No state file

Unlike Terraform, Bicep has no state file to manage or lock. Azure Resource Manager is the source of truth — Bicep just sends the desired state and ARM reconciles it.

06

First-class Azure support

New Azure resource types and API versions appear in Bicep on the same day they land in ARM — no waiting for provider updates like third-party IaC tools require.

Eight Bicep terms you need cold

param

Declares an input value supplied at deploy time. Can have a type (`string`, `int`, `bool`, `object`, `array`), a default, and decorators like `@minLength` or `@allowed`.

Tip: Use `@secure()` decorator on sensitive params — prevents values appearing in deployment logs.

var

Declares an internal computed value reused within the template. Not supplied externally — evaluated at compile time from expressions.

Tip: Use vars to avoid repeating the same string concatenation or condition across multiple resource properties.

resource

Declares an Azure resource. Syntax:

```
resource <symbolicName>  
'<type>@<apiVersion>' = { ... } .
```

The symbolic name is used only within the template to reference the resource.

module

Calls another `.bicep` file as a reusable building block, passing parameters in and receiving outputs back. Enables composition of large environments from tested pieces.

Tip: Modules can reference Bicep Registry files or local paths — use a registry for shared org-wide modules.

→ output

Returns a value from the deployment — resource IDs, connection strings, IP addresses — available to parent templates, pipelines, or subsequent deployments.

Tip: Never output sensitive values (keys, passwords) — outputs appear in deployment history in plain text.

✓ existing

References a resource that already exists in Azure without deploying it. Used to get properties (like a Key Vault ID) from an existing resource to pass to a new one.

Tip: `existing` doesn't create or modify — it's read-only. Great for cross-resource-group references.

↕ targetScope

Sets the deployment scope for the entire file:

```
resourceGroup (default), subscription ,  
managementGroup , or tenant .
```

Tip: Required when deploying at subscription or higher scope — omit it and Bicep assumes resource group.

Bicep Registry

An Azure Container Registry used to store and version shared Bicep modules — teams pull modules by version instead of copying files between repos.

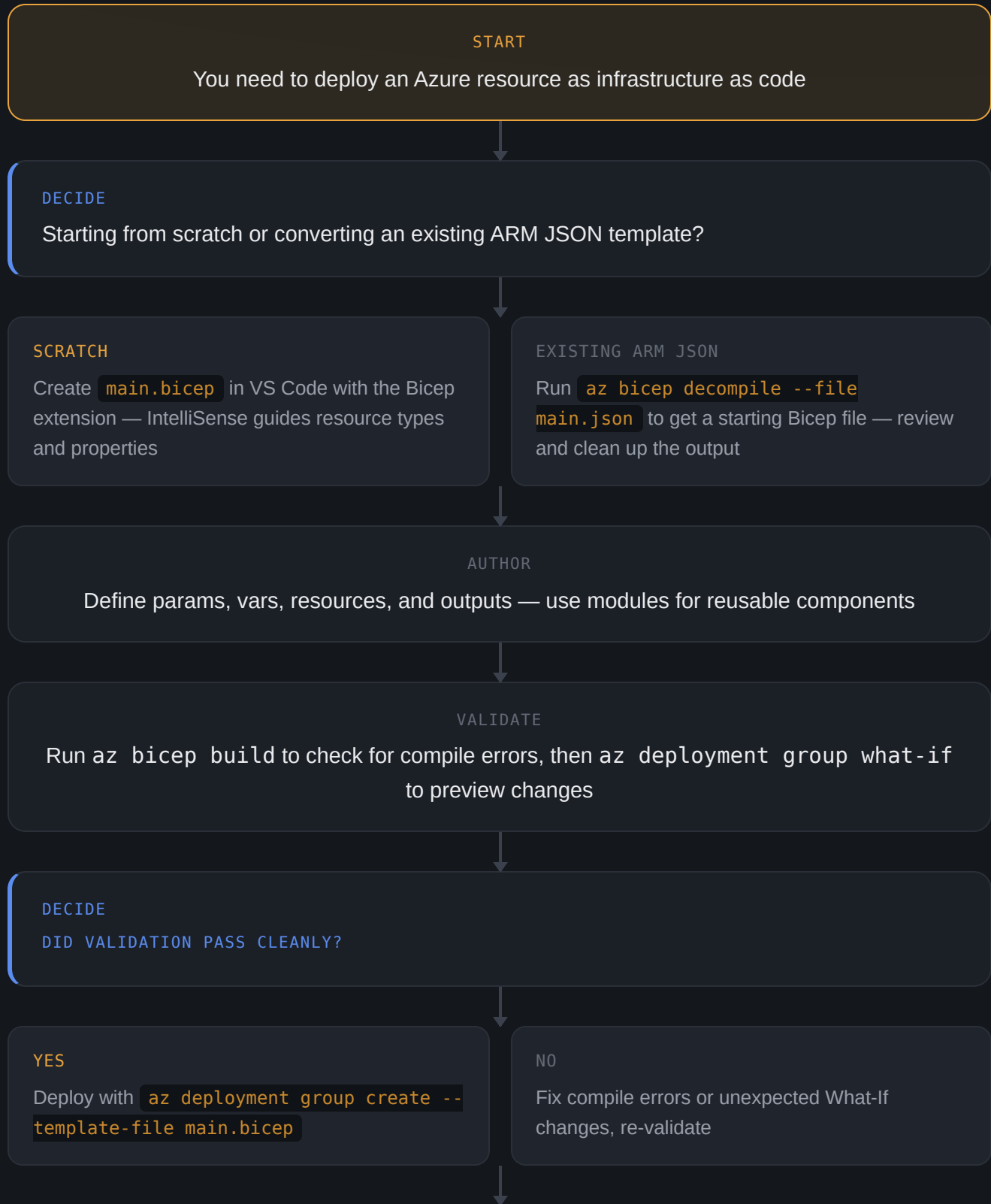
FEATURE	BICEP	ARM JSON	TERRAFORM
Syntax complexity	Low	High	Medium
State file required	No	No	Yes
Day-0 new Azure resource support	Yes	Yes	No — provider lag
Multi-cloud support	No — Azure only	No — Azure only	Yes
Modules / reuse	Yes — native	Linked templates only	Yes
Type safety	Yes — compile time	Runtime only	Partial

KEY MENTAL MODEL

Bicep is **not a separate deployment system** — it is a better way to write ARM templates. `az bicep build` transpiles `.bicep` to `.json`, and that JSON is what Azure actually receives. The Portal, CLI, and pipelines all accept `.bicep` files directly — the transpilation happens transparently.

Bicep authoring and deployment flow

From a new resource to a deployed, tested, and pipeline-ready Bicep template.



DECIDE

Will this be deployed repeatedly (multiple envs or scheduled)?

YES

Commit to Git, add a CI/CD pipeline (GitHub Actions or Azure DevOps) using the `azure/arm-deploy` action

ONE-OFF

CLI deploy is sufficient

DONE

Resources deployed from type-safe, version-controlled, pipeline-ready Bicep

The Bicep toolchain

VS Code + Bicep Extension

BEST FOR: authoring — the only right choice

IntelliSense for every resource type and property, inline error highlighting, resource snippet library, and a dependency graph visualizer. Install once, use for all Bicep work.

```
# Install the Bicep VS Code
extension
code --install-extension ms-
azuretools.vscode-bicep

# Install Bicep CLI (included with
Azure CLI 2.20+)
az bicep install
az bicep upgrade
```

Azure CLI — Bicep commands

BEST FOR: build, decompile, deploy

The primary way to compile, validate, and deploy Bicep from the command line — wrap these in pipeline scripts for full automation.

```
# Compile Bicep → ARM JSON
az bicep build --file main.bicep

# Decompile ARM JSON → Bicep
az bicep decompile --file main.json

# Deploy directly (no manual build
needed)
az deployment group create \
  --resource-group "rg-prod" \
  --template-file "main.bicep" \
  --parameters @params.json

# What-If preview
az deployment group what-if \
  --resource-group "rg-prod" \
  --template-file "main.bicep"
```

🧩 Bicep Modules Pattern

BEST FOR: reusable, composable templates

Break large templates into modules — each module is a `.bicep` file with its own params and outputs. The parent template calls modules like functions.

```
// main.bicep — calling a module
module storage
  './modules/storage.bicep' = {
    name: 'storageDeployment'
    params: {
      storageAccountName:
        'mystorageacct'
      location: location
    }
  }

// Use module output in another
resource
output storageId string =
  storage.outputs.id
```

🌐 Bicep with Key Vault secrets

BEST FOR: injecting secrets at deploy time without hardcoding

Reference Key Vault secrets directly in a parameter file — the secret value is fetched by ARM at deploy time and never stored in the template or logs.

```
// params.json — Key Vault secret
reference
{
  "$schema": "...",
  "parameters": {
    "adminPassword": {
      "reference": {
        "keyVault": {
          "id":
            "/subscriptions/.../vaults/myKV"
        },
        "secretName":
          "vmAdminPassword"
      }
    }
  }
}
```

COMMAND

WHAT IT DOES

`az bicep build`

Compiles `.bicep` → `.json` — validates syntax

`az bicep decompile`

Converts existing ARM JSON to Bicep — starting point for migration

`az deployment group what-if`

Previews changes without deploying

`az deployment group create`

Deploys a `.bicep` or `.json` template to a resource group

`az bicep upgrade`

Updates the Bicep CLI to the latest version

Principles worth internalizing

BICEP IS NOT OPTIONAL FOR AZ-104

The AZ-104 exam tests both Bicep syntax and ARM JSON structure. Know the Bicep equivalents for **param**, **var**, **resource**, **module**, and **output** — and know that `az bicep build` transpiles to JSON, not a separate deployment engine.

NEVER OUTPUT SECRETS

Bicep `output` values appear in the Azure deployment history in **plain text**, accessible to anyone with read access to the resource group. Never output passwords, connection strings, or storage keys — use Key Vault references or managed identities instead.

Six mistakes that show up on the exam

Thinking Bicep is a different deployment engine

Bicep compiles to ARM JSON — it's a better authoring language, not a separate system. ARM is still doing the work.

Outputting sensitive values

Outputs appear in deployment history in plain text. Use `@secure()` on params and Key Vault references — never output keys or passwords.

Forgetting targetScope for subscription-level deployments

Without `targetScope = 'subscription'` at the top of the file, Bicep defaults to resource group scope and the deployment will fail at subscription scope.

Using var for values that need to change per environment

Variables are computed internally — they can't be overridden at deploy time. External inputs that differ per environment belong in `param`, not `var`.

Assuming decompile output is production-ready

`az bicep decompile` produces a usable starting point but often includes redundant expressions, missing decorators, and awkward variable names — always review and clean up.

Not pinning module versions in a registry

Pulling a module without a version tag means the latest is used — a breaking change in the module silently breaks your deployment. Always pin to a specific version.